

Teoria Współbieżności

Laboratorium 2

Zastosowanie semaforów w problemie wyścigu wątków

Kacper Bieniasz

7 października 2024

1 Dane techniczne sprzętu

Obliczenia zostały wykonane na komputerze o następującej specyfikacji:

- Procesor: AMD Ryzen 7 5800U
- Pamięć RAM: 16 GB DDR4 3200 MHz (2×8GB)
- System operacyjny: Windows 11 Home x64

2 Treść zadania

Treść zadania w punktach:

1. Zaimplementować semafor binarny za pomocą metod wait i notify, użyć go do synchronizacji programu Wyścig.
2. Pokazać, że do implementacji semafora za pomocą metod wait i notify nie wystarczy instrukcja if tylko potrzeba użyć while . Wyjaśnić teoretycznie dlaczego i potwierdzić eksperymentem w praktyce. (wskazówka: rozważyć dwie kolejki: czekającą na wejście do monitora obiektu oraz kolejkę związaną z instrukcją wait , rozważyć kto kiedy jest budzony i kiedy następuje wyścig).
3. Zaimplementować semafor licznikowy (ogólny) za pomocą semaforów binarnych. Czy semafor binarny jest szczególnym przypadkiem semafora ogólnego ?

3 Program wykorzystujący semafony binarne do synchronizacji

Kod w języku Java (1) przedstawia wersję programu Wyścig realizującą mechanizm synchronizacji za pomocą semaforów binarnych. Modyfikacja dostarczonego szkieletu programu polegała na dodaniu klasy **Semafor** i wykorzystaniu jej w klasach **IThread** oraz **DThread**. Obie klasy **IThread** i **DThread** mają pole zawierające ten sam semafor. Obie klasy przed wykonaniem swojej operacji na zmiennej współdzielonej wywołują metodę P na semaforze. Po wykonaniu operacji odpowiednio inkrementacji i dekrementacji wywołują metodę V na semaforze. Semafor posiada pole `_stan` mówiące o tym czy semafor jest zajęty czy wolny. Metoda P odpowiada dostęp do zasobu poprzez ewentualne czekania, aż semafor będzie wolny i zajęcie go, wykorzystujemy metodę wait(). Metoda V zwalnia semafor i korzysta z notify() w celu poinformowania pozostałych wątków o zwolnieniu semafora. Zastosowanie takiego mechanizmu synchronizacji daje pewność poprawności otrzymanego wyniku.

```
1 class Semafor {
2     private boolean _stan = true;
3     private int _czeka = 0;
4     public Semafor() {
5     }
6     public synchronized void P() {
```

```

7         while (!_stan) {
8             try {
9                 wait();
10            } catch (InterruptedException e) {
11                Thread.currentThread().interrupt();
12            }
13        }
14        _stan = false;
15    }
16    public synchronized void V() {
17        _stan = true;
18        notify();
19    }
20 }
21 class Counter {
22     private int _val;
23     public Counter(int n) {
24         _val = n;
25     }
26     public void inc() {
27         _val++;
28     }
29     public void dec() {
30         _val--;
31     }
32     public int value() {
33         return _val;
34     }
35 }
36 class IThread extends Thread {
37     private Counter _cnt;
38     private Semafor _sem;
39     public IThread(Counter c, Semafor sem) {
40         _cnt = c;
41         _sem = sem;
42     }
43     public void run() {
44         for (int i = 0; i < 100_000_000; ++i) {
45             _sem.P();
46             _cnt.inc();
47             _sem.V();
48         }
49     }
50 }
51 class DThread extends Thread {
52     private Counter _cnt;
53     private Semafor _sem;
54     public DThread(Counter c, Semafor sem) {
55         _cnt = c;
56         _sem = sem;
57     }
58     public void run() {
59         for (int i = 0; i < 100_000_000; ++i) {
60             _sem.P();
61             _cnt.dec();
62             _sem.V();
63         }
64     }
65 }
66 class Race2 {
67     public static void main(String[] args) {
68         Counter cnt = new Counter(0);

```

```

69     Semafor sem = new Semafor();
70     IThread it = new IThread(cnt, sem);
71     DThread dt = new DThread(cnt, sem);
72     it.start();
73     dt.start();
74     try {
75         it.join();
76         dt.join();
77     } catch (InterruptedException ie) {
78     }
79     System.out.println("value=" + cnt.value());
80 }
81 }

```

Kod Java 1: Wersja z semaforem binarnym

4 Implementacja semafora z wykorzystaniem instrukcji if zamiast while

Wykorzystanie while zamiast if przy implementacji semafora za pomocą metod wait() i notify() jest konieczne ze względu na możliwość wystąpienia wyścigu i nieprzewidywalne zachowanie w systemie wielowątkowym. Gdy wątek wchodzi do metody zsynchronizowanej, zyskuje dostęp do monitora obiektu. Pozostałe wątki próbujące wejść do tego monitora są blokowane w kolejce monitorowej i muszą czekać na zwolnienie monitora. Gdy jeden z wątków wywołuje metodę wait(), zwalnia monitor obiektu, ale zostaje umieszczony w kolejce czekającej na spełnienie warunku (wait queue). Kiedy wątek wchodzi w stan wait(), inny wątek może przejąć monitor i pracować nad nim. Jeśli ten inny wątek wywoła notify(), budzi pierwszy wątek (z kolejki wait), który zostaje przeniesiony z powrotem do kolejki monitorowej. Jednak zanim wątek obudzony faktycznie odzyska kontrolę nad monitorem, inny wątek może ponownie wejść do monitora i zmienić stan obiektu. W związku z tym, budzenie wątku przez notify() nie gwarantuje, że obudzony wątek natychmiast otrzyma dostęp do monitora. Dlatego, jeśli użyjemy instrukcji if, która sprawdza tylko raz warunek, istnieje ryzyko, że obudzony wątek nie znajdzie się w odpowiednim stanie obiektu.

Zamieniając instrukcje while na if w metodzie P() semafora i uruchamiając program kilkakrotnie otrzymane wyniki zawsze były niepoprawne. Oto kilka przykładowych wartości zmiennej współdzielonej w programie wykorzystującym if:

964
1093
657
383
2
244
466

Tabela 1: Wyniki programu wykorzystującego instrukcji if zamiast while

5 Implementacja semafora licznikowego

Semafor binary jest szczególnym przypadkiem semafora licznikowego ponieważ, przechowuje dwie wartości 0 i 1. W przypadku semafora licznikowego może on przyjmować więcej wartości. Kod (2) w języku Java zawiera implementację semafora licznikowego. Klasa BinSemafor realizuje podstawowy semafor binarny. Natomiast klasa LiczSemafor realizuje semafor licznikowy. Jego działanie wygląda następująco:

- Przechowuje liczbę dostępnych zasobów, zmienna `_count`.
- W polu `mutex` przechowuje semafor binarny w celu ochrony licznika.
- Zajmowanie zasobu realizuje metoda `P()` i sposób jej działania wygląda następująco:
 1. Wejście do sekcji krytycznej z wykorzystaniem metody `P()` na semaforze binarnym.
 2. W `while` sprawdzamy ilość dostępnych zasobów.
 3. Zwalniamy `mutex` w celu umożliwienia innym wątkom dostępu do licznika.
 4. Oczekujemy, aż zasób będzie dostępny.
 5. Jeśli jakiś zasób będzie dostępny to ponownie zajmujemy `mutex`.
 6. Zmniejszamy licznik o jeden, czyli zajmujemy zasób i wychodzimy z sekcji krytycznej metodą `V()` semafora binarnego.
- Zwalnianie realizuje metoda `V()` i sposób jej działania wygląda następująco:
 1. Wchodzimy do sekcji krytycznej.
 2. Zwiększamy licznik zasobów.
 3. Powiadamy pozostałe wątki.
 4. Wychodzimy z sekcji krytycznej.

```

1  class BinSemafor {
2      private boolean _stan = true;
3      public synchronized void P() {
4          while (!_stan) {
5              try {
6                  wait();
7              } catch (InterruptedException e) {
8                  Thread.currentThread().interrupt();
9              }
10         }
11         _stan = false;
12     }
13     public synchronized void V() {
14         _stan = true;
15         notify();
16     }
17 }
18 class LiczSemafor {
19     private int _count;
20     private BinSemafor mutex = new BinSemafor();
21     public LiczSemafor(int initialCount) {
22         _count = initialCount;
23     }
24
25     public void P() {
26         mutex.P();
27         while (_count == 0) {
28             try {
29                 mutex.V();
30                 synchronized (this) {
31                     wait();
32                 }
33                 mutex.P();
34             } catch (InterruptedException e) {
35                 Thread.currentThread().interrupt();
36             }
37         }
38         _count--;

```

```

39     mutex.V();
40 }
41 public void V() {
42     mutex.P();
43     _count++;
44     synchronized (this) {
45         notify();
46     }
47     mutex.V();
48 }
49 }

```

Kod Java 2: Semafor licznikowy zaimplementowany za pomocą semafora binarnego

6 Wnioski

Synchronizacja wątków za pomocą semaforów jest kluczowym elementem w programowaniu wielowątkowym, gdy istnieje potrzeba zarządzania współdzielonymi zasobami. Niewłaściwe zastosowanie metod synchronizacji, takich jak użycie `if` zamiast `while` w blokowaniu wątków, może prowadzić do poważnych błędów, takich jak wyścigi wątków lub monopolizacja zasobów przez jeden wątek. Implementacja semafora licznikowego za pomocą semaforów binarnych pokazuje, jak można synchronizować dostęp do większej liczby zasobów w sposób bezpieczny i efektywny.

Literatura

- [1] Per Brinch Hansen: Java Insecure Parallelism. ACM SIGPLAN Notices, April 1999, s. 38-45.
- [2] Thomas W. Christopher, George K. Thiruvathukal: High-Performance Java Platform Computing, Prentice Hall, Luty 2001.